

StockDex — ALM

Développement

Git workflow · Conventions · Pipeline CI

Jeremy Lardet © 2026

1. README — Installation & démarrage

Prérequis

- Node.js $\geq 20.x$
- npm $\geq 10.x$
- Git

Installation

```
git clone https://github.com/<org>/stockdex && cd stockdex
```

```
npm install # installe les workspaces client et server
```

```
cp packages/server/.env.example packages/server/.env # renseigner JWT_SECRET
```

Lancer les tests

```
npm test # depuis la racine — lance les tests Jest du workspace server
```

Démarrer l'application

```
npm run dev # lance Express (port 3000) + ouvre packages/client/index.html
```

Le frontend est une SPA statique servie directement (pas de bundler en dev).

2. Workflow Git

Stratégie retenue : GitHub Flow — branche main toujours déployable, feature branches courtes.

Structure des branches

- main — production, protégée, merge via PR uniquement
- feature/<slug> — nouvelle fonctionnalité, durée max 1 jour
- fix/<slug> — correction de bug
- chore/<slug> — maintenance (CI, dépendances, docs)

Cycle de vie d'une feature branch

1. git checkout -b feature/trade-proposal
2. Développement + commits atomiques
3. Push + ouverture d'une Pull Request vers main
4. CI s'exécute automatiquement (lint, tests, build)
5. Review par un pair (ou auto-review si solo)
6. Merge squash → main, suppression de la branche

Justification : GitHub Flow est adapté au déploiement continu sur Render/Vercel. La simplicité est préférable à GitFlow pour une équipe de 3-4 personnes sur une semaine.

3. Conventions de commit — Conventional Commits

Format : <type>(<scope>): <description courte en impératif, en anglais>

Commit	Description
feat(auth)	add JWT login endpoint with bcrypt password hashing
feat(boosters)	implement weighted card draw by currentRarity
feat(trade)	add trade proposal and acceptance flow
feat(market)	add nodecron job for price fluctuation every 30s
feat(socket)	broadcast price updates via Socket.IO per card family
fix(trade)	prevent trade with own cards — validation on POST /trades
fix(market)	correct rarity threshold boundary ($\geq$, not $>$)
refactor(db)	extract db init into dedicated module, add WAL mode
test(boosters)	add unit tests for drawCards weight distribution
chore(ci)	add GitHub Actions workflow for lint + test + build

Types utilisés : feat, fix, refactor, test, chore, docs, perf. Scope = module applicatif (auth, boosters, trade, market, socket, db, ci).

4. Pipeline CI — GitHub Actions

Déclencheurs

- Push sur toute branche feature/*
- Ouverture / mise à jour d'une Pull Request vers main

Étapes du pipeline

- 1. Checkout du dépôt
- 2. Setup Node.js 20.x
- 3. npm ci — installation déterministe des dépendances
- 4. Lint — ESLint sur packages/server/src et packages/client/src
- 5. Tests unitaires — Jest avec couverture (--coverage)
- 6. Build — vérification que le serveur démarre sans erreur (node --check)

Le pipeline échoue si une étape renvoie un code non-nul. Aucun merge vers main n'est autorisé si le CI est rouge.

5. Usage de l'IA dans le développement

Outil utilisé : Claude Code (terminal) + GitHub Copilot (VSCode inline).

- Génération des migrations SQLite et des requêtes bettersqlite3

- Scaffolding du composant React StockCard avec les props TypeScript
- Suggestion du pattern de test Jest pour le moteur de fluctuation

Impact sur le flux de travail : réduction du temps de boilerplate estimée à ~40%. L'IA a été supervisée pour chaque suggestion — les requêtes SQL ont été relues manuellement avant intégration.